

Звіт з лабораторної роботи

1. Тема та мета

Тема: Code Review та рефакторинг коду.

Мета: навчитись проводити взаємне рецензування коду, виявляти проблеми якості, документувати їх та виконувати рефакторинг з підтвердженням регресійними тестами.

Виконала: Софія (група ПЗПІ-25-1)

Репозиторій: <https://github.com/AnnaYevsovych/CratlogGroup4>

2. Результати Code Review

№	Файл / Рядок	Проблема	Категорія	Рекомендація
1	Work.java, p.30	getTags() захищає список копією, але removeTag() відсутній	Inconsistent encapsulation	Додати метод removeTag(String tag)
2	Creator.java, p.22	String.matches() компілює regex при кожному виклику	Performance smell	Використати Pattern.compile() один раз
3	Portfolio.java, p.55	stream().anyMatch() для перевірки дублікатів — O(n) при кожному додаванні	Wrong data structure	Використати Set<Integer> workIds для O(1) перевірки
4	PortfolioTest.java	Немає тесту на ліміт MAX_WORKS=50 (BVA)	Missing BVA	Додати TC-P19: 50 робіт -> 51-а кидає IllegalStateException
5	Creator.java, p.24	username не валідується в конструкторі — можна передати null	Missing validation	Додати перевірку: if (username == null username.isBlank()) throw

3. Посилання на PR

PR: буде додано після створення гілки

<https://github.com/AnnaYevsovych/CratlogGroup4/pull/6>

4. Результати рефакторингу

Рефакторинг 1: Додано final до поля workId (Work.java)

БУЛО:

```
private int workId;
```

СТАЛО:

```
private final int workId;
```

ЧОМУ: Ідентифікатор роботи не повинен змінюватись після створення об'єкта. `final` гарантує immutability.

Рефакторинг 2: Замінено `String.matches()` на `Pattern.compile()` (`Creator.java`)

БУЛО:

```
email.trim().matches(EMAIL_REGEX)
```

СТАЛО:

```
EMAIL_PATTERN.matcher(email.trim()).matches() // + додано: private
static final Pattern EMAIL_PATTERN = Pattern.compile(EMAIL_REGEX);
```

ЧОМУ: `String.matches()` компілює regex при кожному виклику. `Pattern.compile()` робить це один раз — ефективніше.

Рефакторинг 3: Додано валідацію `username` (`Creator.java`)

БУЛО:

```
this.username = username;
```

СТАЛО:

```
if (username == null || username.isBlank()) {      throw new
IllegalArgumentException("Username не може бути порожнім"); }
this.username = username;
```

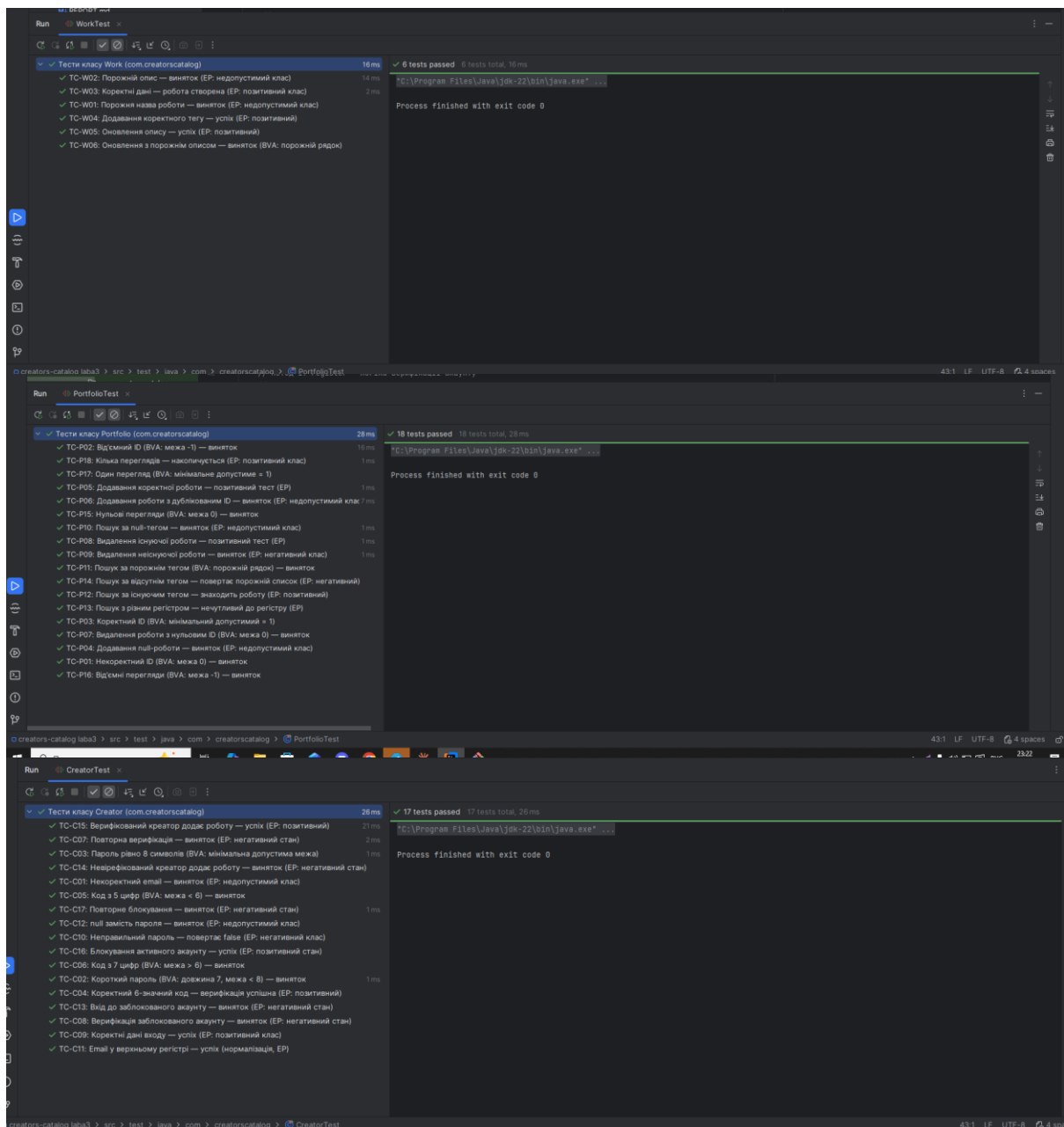
ЧОМУ: Конструктор валідував email і пароль, але не `username`. Додана перевірка усуває можливий `NullPointerException`.

5. Регресійне тестування

Після рефакторингу запущено всі JUnit-тести:

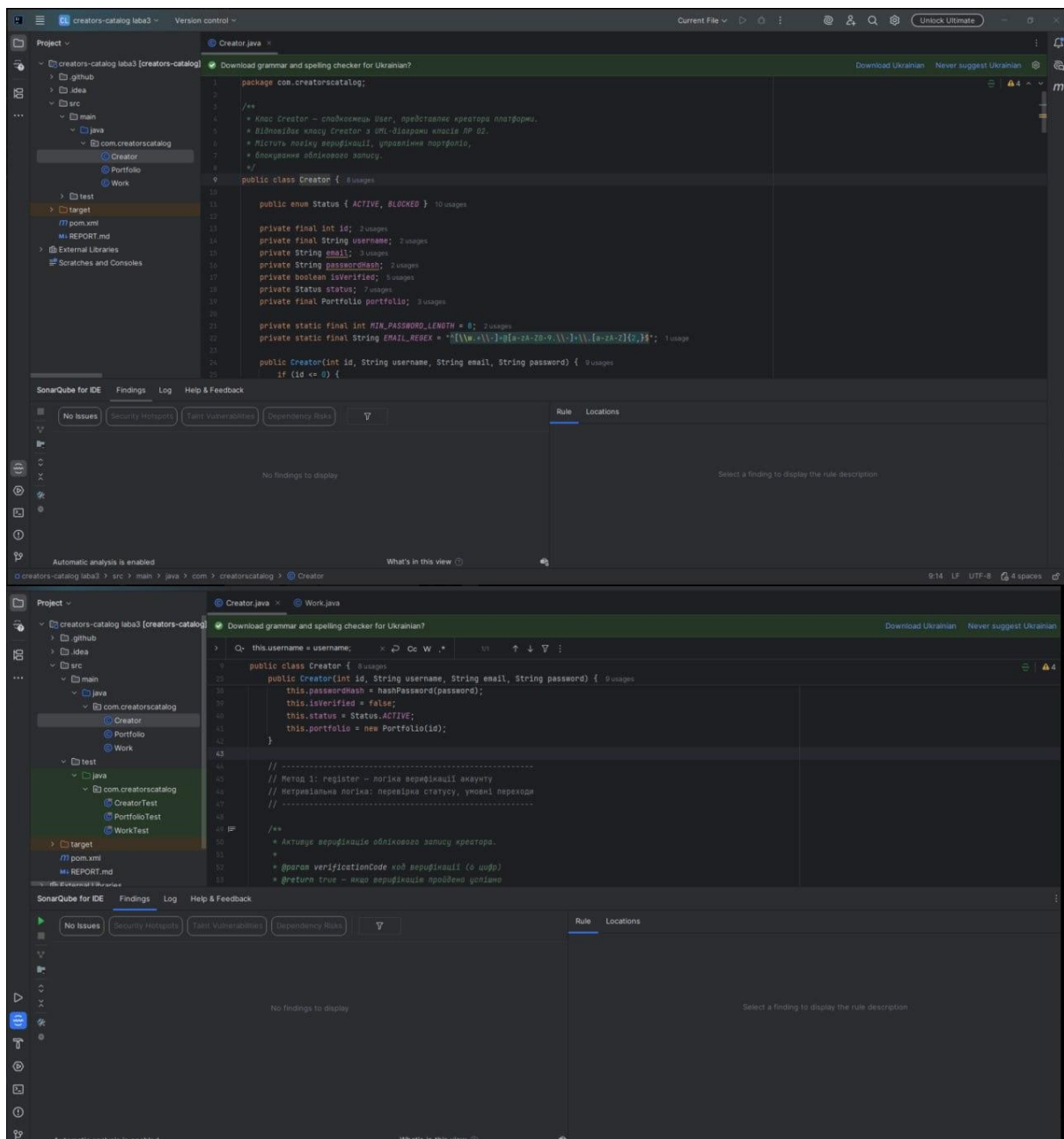
- `CreatorTest`: 17 tests passed
- `PortfolioTest`: 18 tests passed
- `WorkTest`: 6 tests passed

Всього: 41 тест — усі GREEN. Process finished with exit code 0.



6. Порівняння метрик ДО / ПІСЛЯ

Метрика	ДО	ПІСЛЯ
SonarLint issues	0	0
Кількість тестів	41	41
Всі тести green	Так	Так
Валідація username	Відсутня	Додана
Regex компіляція	При кожному виклику	Один раз (Pattern.compile)
Поле world	Mutable (int)	Immutable (final int)



7. Підсумкова рефлексія

Найціннішим досвідом під час Code Review стало виявлення проблем з продуктивністю — регекс рескомпіляція та $O(n)$ пошук дублікатів — які не впливають на коректність але важливі в реальних системах. Я зрозуміла що хороший код це не лише правильна логіка, а й ефективність та розширюваність. Рефакторинг з регресійним тестуванням (41 тест green) показав що зміни можна вносити впевнено при наявності тестового покриття.